

Experiment 3: Creation of a Simple Network Topology Using Open-Source Network Virtualization Tools

Objective

To create and configure simple network topologies using **Mininet** and verify communication between virtual network nodes using basic Linux networking commands.

Tools Used

- Ubuntu Linux
 - VirtualBox
 - Mininet
 - Open vSwitch
-

1. Installation and Setup of Mininet

Figure 1: Installation and initialization of Mininet

Description:

Mininet was installed on the Ubuntu virtual machine using the apt package manager along with its required dependencies. The installed Mininet version was verified, and the Mininet CLI was successfully initialized, confirming that the environment was ready for creating virtual network topologies.

Commands:

```
sudo apt install mininet -y
```

```
mn --version
```

```
sudo mn
```

2. Single-Switch Topology

Figure 2: Creation and verification of a single-switch topology

Description:

A simple Mininet topology consisting of **four hosts (h1, h2, h3, h4)** connected to a single switch **s1** was created using the single topology. The nodes and links commands were used to verify the available nodes and their connections. The pingall command confirmed communication between all hosts with successful packet delivery.

```
Sep 19 21:05
debasmita-bose@debasmita-bose-VirtualBox: ~
debasmita-bose@debasmita-bose-VirtualBox:~$ sudo apt install mininet -y
Installing:
  mininet

Installing dependencies:
  cgroup-tools  libcgroup3  libunbound8  net-tools  openvswitch-switch  python3-openvswitch  python3-sortedcontainers
  iperf         libevent-2.1-7t64  libxdp1      openvswitch-common  python3-netifaces  python3-packaging  socat

Suggested packages:
  openvswitch-doc  python3-graphviz  python3-unbound  python-sortedcontainers-doc

Summary:
  Upgrading: 0, Installing: 15, Removing: 0, Not Upgrading: 379
  Download size: 5,625 kB
  Space needed: 19.2 MB / 13.0 GB available

Get:1 http://archive.ubuntu.com/ubuntu resolute/universe amd64 libcgroup3 amd64 3.1.0-2build1 [63.0 kB]
Get:2 http://archive.ubuntu.com/ubuntu resolute/universe amd64 cgroup-tools amd64 3.1.0-2build1 [85.6 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute/universe amd64 iperf amd64 2.2.1+dfsg-1build1 [161 kB]
Get:4 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 libevent-2.1-7t64 amd64 2.1.12-stable-10ubuntu0.1 [145 kB]
Get:5 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 libunbound8 amd64 1.24.2-1ubuntu2.2 [456 kB]
Get:6 http://archive.ubuntu.com/ubuntu resolute/main amd64 libxdp1 amd64 1.6.2-1ubuntu1 [65.7 kB]
Get:7 http://archive.ubuntu.com/ubuntu resolute/main amd64 net-tools amd64 2.10-2ubuntu1 [151 kB]
Get:8 http://archive.ubuntu.com/ubuntu resolute/main amd64 python3-netifaces amd64 0.11.0-2build7 [17.9 kB]
Get:9 http://archive.ubuntu.com/ubuntu resolute/main amd64 python3-packaging all 26.0-1 [58.7 kB]
Get:10 http://archive.ubuntu.com/ubuntu resolute/main amd64 python3-sortedcontainers all 2.4.0-2build1 [27.1 kB]
Get:11 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 socat amd64 1.8.1.1-1ubuntu0.1 [403 kB]
Get:12 http://archive.ubuntu.com/ubuntu resolute/universe amd64 mininet amd64 2.3.0-1.1build1 [131 kB]
Get:13 http://archive.ubuntu.com/ubuntu resolute/main amd64 openvswitch-common amd64 3.7.1-2 [1,081 kB]
Get:14 http://archive.ubuntu.com/ubuntu resolute/main amd64 openvswitch-switch amd64 3.7.1-2 [1,683 kB]
Get:15 http://archive.ubuntu.com/ubuntu resolute/main amd64 python3-openvswitch amd64 3.7.1-2 [1,097 kB]
Fetched 5,625 kB in 5s (1,193 kB/s)
Selecting previously unselected package libcgroup3:amd64.
(Reading database... 150898 files and directories currently installed.)
Preparing to unpack .../00-libcgroup3_3.1.0-2build1_amd64.deb...
Unpacking libcgroup3:amd64 (3.1.0-2build1)...
Selecting previously unselected package cgroup-tools.
Preparing to unpack .../01-cgroup-tools_3.1.0-2build1_amd64.deb...
Unpacking cgroup-tools (3.1.0-2build1)...
Selecting previously unselected package iperf.
```

Command:

```
sudo mn --topo single,4
```

Verification commands:

```
nodes
```

```
links
```

```
pingall
```

Result:

All four hosts were reachable, with **0% packet loss (12/12 packets received)**.

3. Tree Topology

Figure 3: Creation and connectivity verification of a tree topology

Description:

A tree-based Mininet topology was created using **four hosts and three switches** with a fanout of two. The topology was initialized using the tree option, and the nodes and links commands were used to verify the network structure. The IP configuration and routing information of individual hosts were also examined. Connectivity between hosts was verified using the ping command.

Command:

```
debasmita-bose@debasmita-bose-VirtualBox: ~ -- sudo mn --topo single,4
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller

*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> nodes
available nodes are:
h1 h2 h3 h4 s1
mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s1-eth2 (OK OK)
h3-eth0<->s1-eth3 (OK OK)
h4-eth0<->s1-eth4 (OK OK)
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
h4 h4-eth0:s1-eth4
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0 s1-eth4:h4-eth0
mininet> ping all
*** Unknown command: ping all
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
```

`sudo mn --topo tree,depth=2,fanout=2`

Verification commands:

`nodes`

`links`

`h2 ip addr`

`h3 ip addr`

`h2 route`

`h2 ping -c 4 h3`

Result:

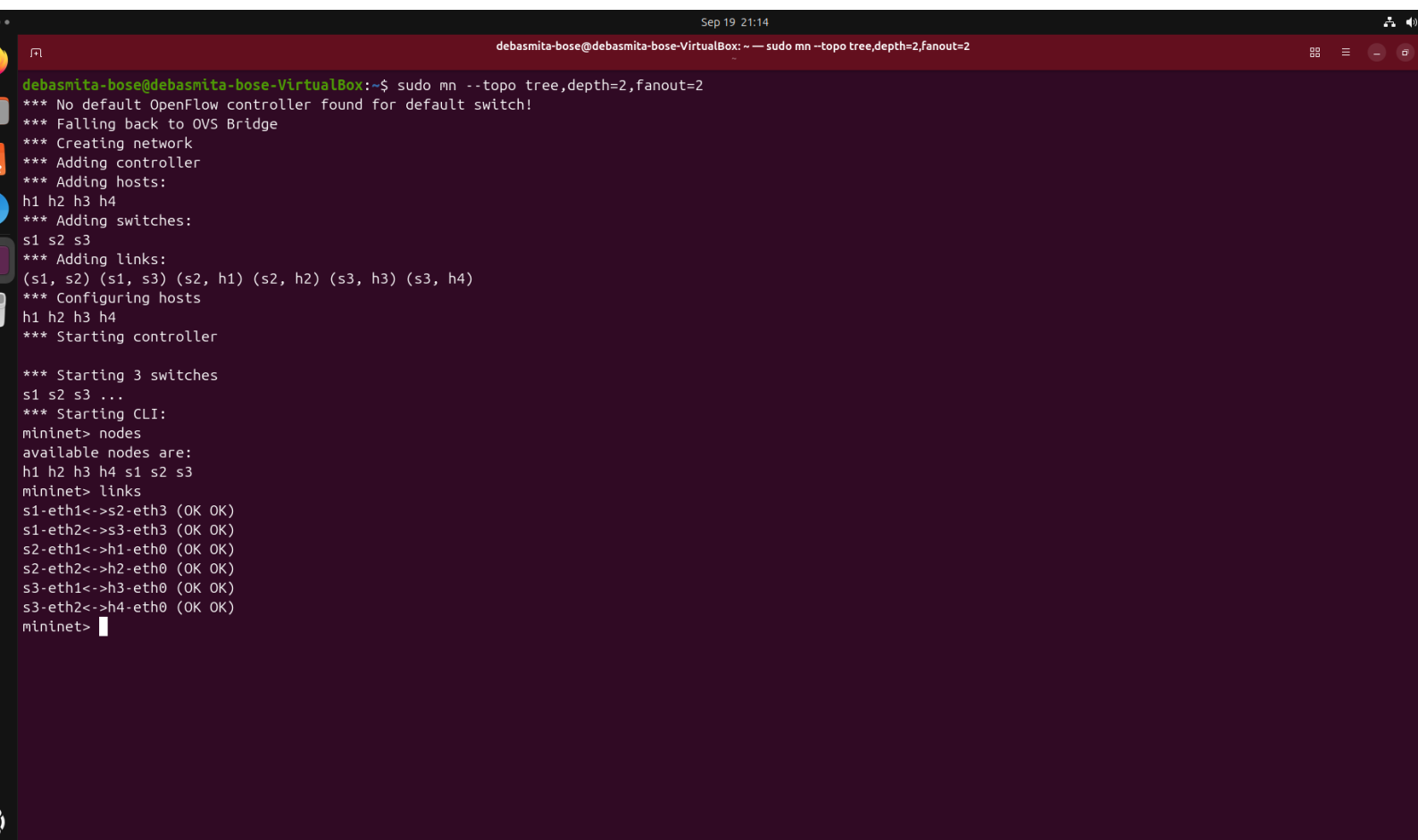
Host **h2 (10.0.0.2)** successfully communicated with **h3 (10.0.0.3)**, receiving **4/4 packets with 0% packet loss**.

4. Linear Topology

Figure 4: Creation and verification of a linear network topology

Description:

A linear Mininet topology consisting of **four hosts and four switches** was created using the linear topology. The nodes and links commands were used to verify the topology and its connections. The



```
debasmita-bose@debasmita-bose-VirtualBox: ~ — sudo mn --topo tree,depth=2,fanout=2
debasmita-bose@debasmita-bose-VirtualBox:~$ sudo mn --topo tree,depth=2,fanout=2
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3
*** Adding links:
(s1, s2) (s1, s3) (s2, h1) (s2, h2) (s3, h3) (s3, h4)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller

*** Starting 3 switches
s1 s2 s3 ...
*** Starting CLI:
mininet> nodes
available nodes are:
h1 h2 h3 h4 s1 s2 s3
mininet> links
s1-eth1<->s2-eth3 (OK OK)
s1-eth2<->s3-eth3 (OK OK)
s2-eth1<->h1-eth0 (OK OK)
s2-eth2<->h2-eth0 (OK OK)
s3-eth1<->h3-eth0 (OK OK)
s3-eth2<->h4-eth0 (OK OK)
mininet> 
```

pingall command was then executed to test end-to-end connectivity among all hosts. The IP address and routing table of **h2** were also inspected, followed by a direct ping test to **h4**.

Command:

```
sudo mn --topo linear,4
```

Verification commands:

```
nodes
```

```
links
```

```
pingall
```

h2 route

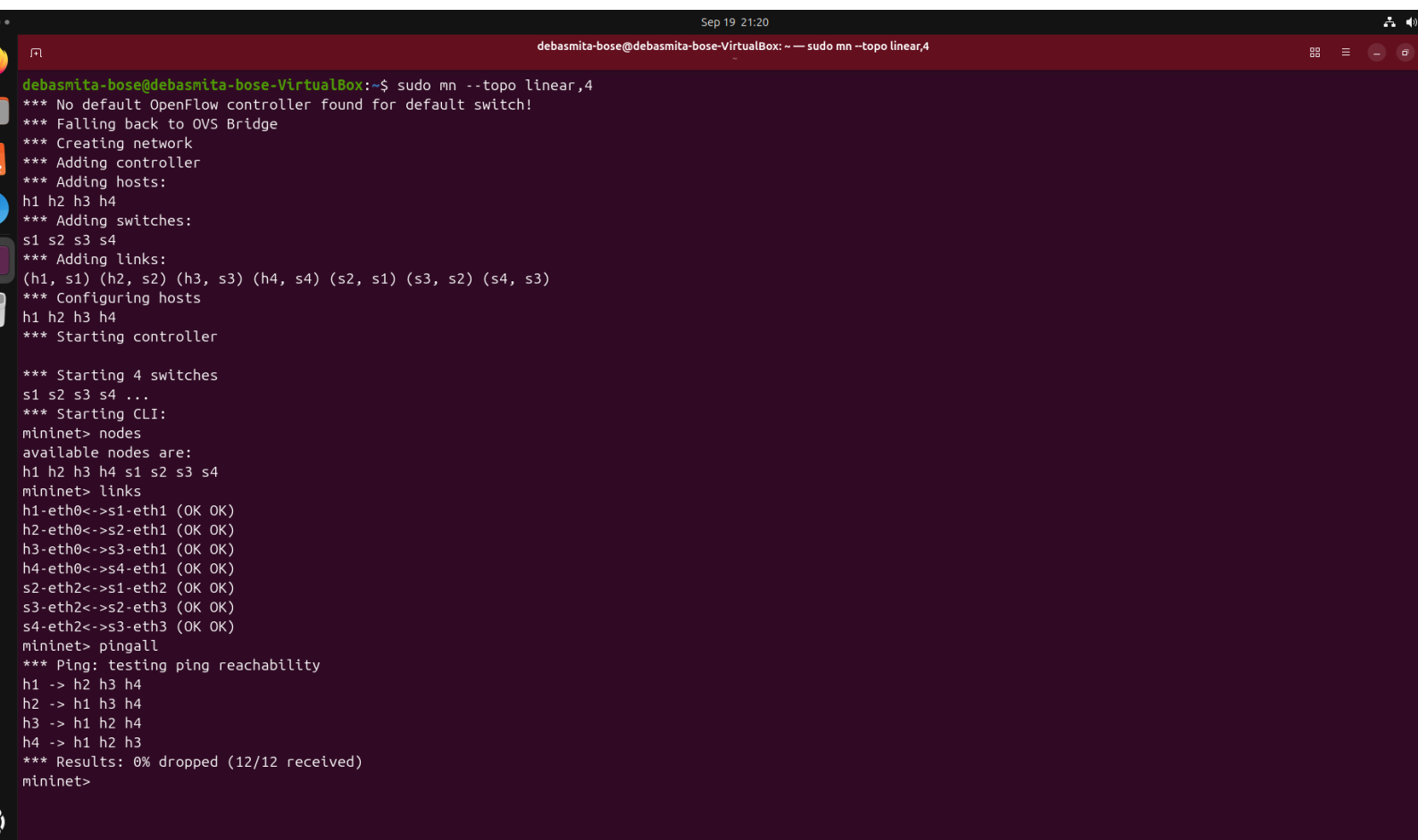
h2 ip addr

h2 ping -c 6 h4

Result:

All hosts successfully communicated with each other, with **0% dropped packets (12/12 received)**.
The direct connectivity test from **h2 (10.0.0.2)** to **h4 (10.0.0.4)** also resulted in **0% packet loss**.

5. Overall Result



```
debasmita-bose@debasmita-bose-VirtualBox: ~ -- sudo mn --topo linear,4
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3 s4
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (h4, s4) (s2, s1) (s3, s2) (s4, s3)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller

*** Starting 4 switches
s1 s2 s3 s4 ...
*** Starting CLI:
mininet> nodes
available nodes are:
h1 h2 h3 h4 s1 s2 s3 s4
mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s2-eth1 (OK OK)
h3-eth0<->s3-eth1 (OK OK)
h4-eth0<->s4-eth1 (OK OK)
s2-eth2<->s1-eth2 (OK OK)
s3-eth2<->s2-eth3 (OK OK)
s4-eth2<->s3-eth3 (OK OK)
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet>
```

Figure 5: Verification of Mininet network topologies and host connectivity

Description:

Different network topologies were successfully created and tested using Mininet. The **single-switch, tree, and linear topologies** were constructed with different numbers of hosts and switches. Network structure was verified using nodes and links, while IP configuration and routing information were examined using ip addr and route. Connectivity between the virtual hosts was successfully verified using ping and pingall, with the demonstrated tests showing **0% packet loss**.